

INSTITUT CURIE · PARIS · 17-18 SEPTEMBER 2026

Hacking Cancer Research with LLM Agents & the Model Context Protocol

Day 1 · opening session

Elliott Jacopin, RIKEN · Laurence Calzone, Institut Curie

Who is in the room

17 responses to the application form

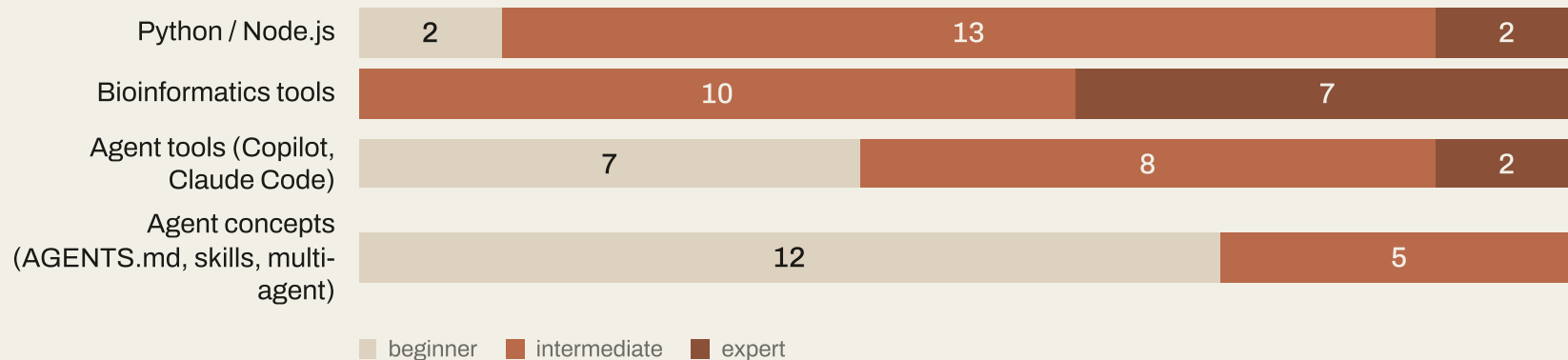
CAREER STAGE

- 6 staff scientists
- 4 PhD students
- 4 postdocs
- 3 an engineer, an MD, a master's student

PROFILE

- 11 bioinformaticians / computational biologists
- 3 software engineers / data scientists
- 2 experimental biologists
- 1 clinical researcher

SELF-RATED



What you want to plug in

From the same 17 answers.

WHAT THE AGENT SHOULD REACH

Bioinformatics and systems-biology tools	15
Public databases: TCGA, COSMIC, ClinVar, GEO	10
Literature and knowledge retrieval	10
Internal institutional databases	4

THE FIVE TABLES

T1	Wrapping simulation & modelling engines	2
	Boolean model construction, MaBoSS, PhysiCell	
T2	Wrapping imaging & chromatin-architecture pipelines	4
	Hi-C, DNA/RNA-FISH, Hi-M design, microscopy analysis	
T3	Wrapping genomics & omics analysis tools	6
	Snaptron, immune-cell and proteomics pipelines, multi-omics	
T4	Exposing institutional data, cohorts & tooling	2
	internal databases, cohort queries, metadata mapping	
T5	Composing existing servers for a clinical question	3
	tumour microenvironment stratification, CAR-T persistence	

The right-hand count is how many of your stated topics fall on that table.

An afternoon to converge

13:00 – 13:30 Welcome & short introductions

13:30 – 15:00 Introduction to MCP for biology

This session: reminders, plugins, and the hands-on examples

15:00 – 15:30 Coffee break

15:30 – 18:00 Group formation & guided brainstorming

Thematic tables; organizers rotate. Converge on a project and on which server each of you writes.

from 18:00 Cocktail

A full day to build

09:00 – 09:15 Coffee & recap of group projects

09:15 – 12:30 Hacking session 1

Each group implements their MCP server(s)

12:30 – 13:30 Lunch

13:30 – 16:30 Hacking session 2

16:30 – 17:00 Coffee & demo prep

17:00 – 18:00 Group presentations & demos

7 minutes each, 3 minutes of questions

18:00 – 18:30 Closing · next steps for the joint paper

The examples for today

Two marketplaces. Add both: everything you install this afternoon is in one of them.

WRITTEN FOR TODAY

The hackathon repository

github.com/sysbio-curie/MCP_Hackaton

A suite of deliberately easy examples: five one-feature servers, a one-file skill, the server inventory at `inventory/mcp-servers.md`. Each small enough to read in a minute — and from tomorrow, your projects sit beside them.

IN DAILY USE

Nest

github.com/CrackingShells/Nest

My own catalogue, and none of it was written to teach: the report, roadmap and git-discipline skills, the plugin generator, the search server. Same format, real-world size.

GOTCHA

A marketplace is a git repository with a manifest.

Adding agent plugins

Both sources from the last slide, added the same way. A marketplace is a git repository, so its address is just `<owner>/<repo>`; a plugin is named for the marketplace it came from.

Claude Code

PATTERN

```
claude plugin marketplace add  
  <owner>/<repo>  
claude plugin install  
  <plugin>@<marketplace>
```

EXAMPLE

```
claude plugin marketplace add  
  sysbio-curie/MCP_Hackaton  
claude plugin install  
  demo-tools@MCP_Hackaton
```

Codex

PATTERN

```
codex plugin marketplace add  
  <owner>/<repo>  
codex plugin add  
  <plugin>@<marketplace>
```

EXAMPLE

```
codex plugin marketplace add  
  CrackingShells/Nest  
codex plugin add  
  writing-reports@cracking-shells
```

VS Code

PATTERN

No marketplace command. Four steps, from the command palette:

- 1 cmd/ctrl+shift+p
- 2 Type: Chat: Install Plugins from Source
- 3 add <owner>/<repo>
- 4 select the plugin in the GUI

This session

- | | | |
|---|---|--------|
| 1 | Humans are also agents
so you are already in a multi-agent system | 10 min |
| 2 | $n_H : n_{\text{lema}}$
patterns and gotchas following the <i>human-to-lema</i> ratio | 10 min |
| 3 | Working with lemas
agent files, skills, servers, hooks, delegation, the record | 45 min |
| 4 | The hackathon
what exists for biology, how to partition, how to choose, and how you present | 20 min |

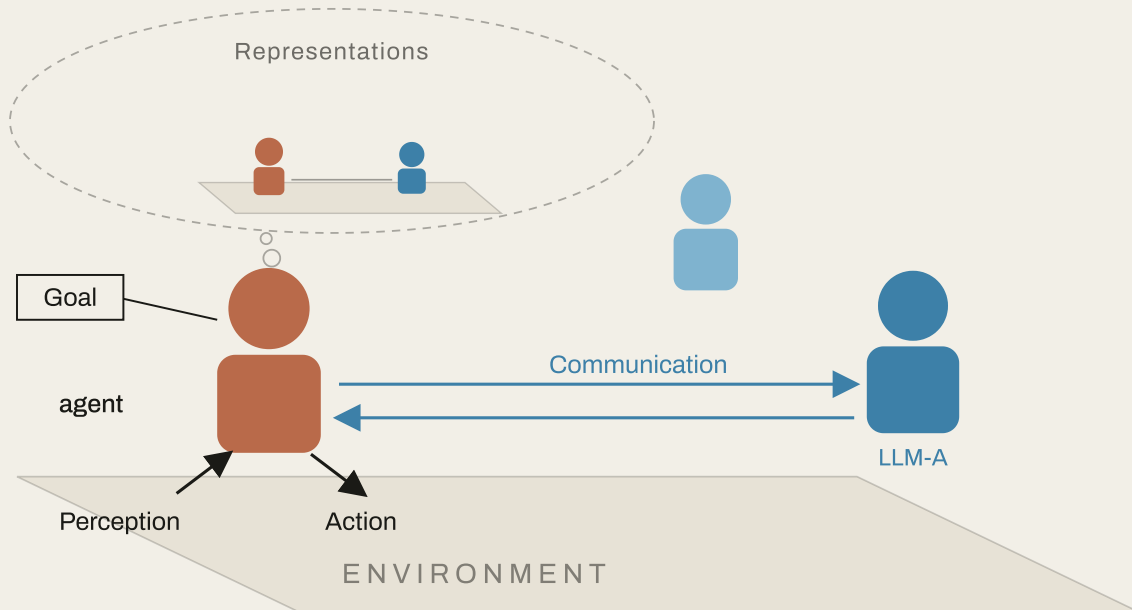
PART 1

Humans are also agents

You are inside the system you are building

The human as an agent

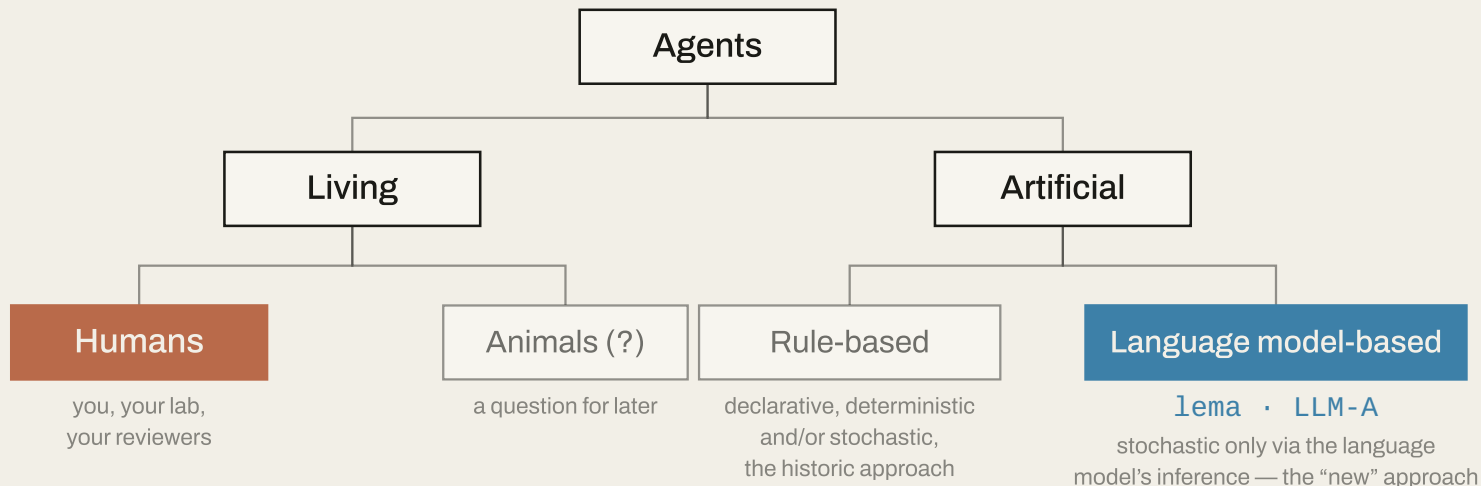
An agent perceives its environment and acts on it toward a goal. Nothing in that says “software”.



■ humans, throughout this deck ■ lemas, throughout this deck

After Ferber, 1999.

Where a lema sits



GOTCHA

Avoid thinking about your lema in isolation: it is always at least two agents, you and the lema.

PART 2

$$n_H : n_{\text{lema}}$$

How many of you, how many of them

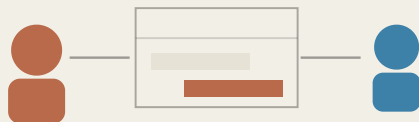
1:1

one of you, one of them

The same ratio buys two entirely different days. The axis that separates them is not the count, it is whether you are watching.

INTERACTIVE

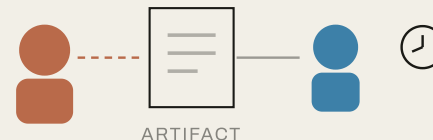
You watch it work



High bandwidth. Think of the lema as an enhanced keyboard: the effort moves from doing the work to watching it. Scales as far as your attention, and no further.

ASYNCHRONOUS

It runs, you read the result



Low bandwidth, on a clock. 5 to 30 minutes a cycle, reading figures and questions.

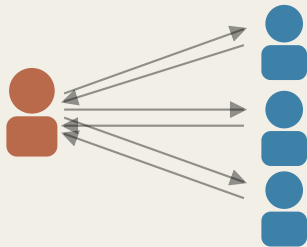
1:n

one of you, several of them

Where most of you already are, without having decided to be. Raising the count combines with either pattern, and changes what each one costs.

INTERACTIVE

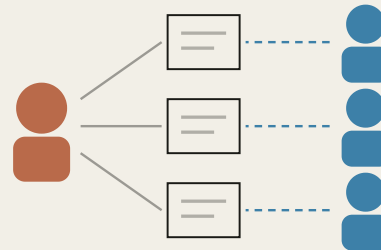
You cycle between them



Simultaneous play. Several live conversations, and you switch between all of them. Your capacity to focus is the main parameter deciding the upper bound on how many parallel sessions you can run.

ASYNCHRONOUS

They run, you read the pile



A team leader's morning. Activity reports come in; you review, you comment, you reject, you validate. They carry on from your feedback, and you never watch them work.

$n_H : n_{\text{lema}}$

the general form

Every arrangement is a value of this ratio.

ARRANGEMENT	$n_H : n_{\text{lema}}$	WHAT IT COSTS YOU
You watch it work INTERACTIVE	1 : 1	your whole attention, live
You read what it produced ASYNCHRONOUS	1 : 1	5 to 30 minutes a cycle
You run several at once INTERACTIVE	1 : n	you become the integration layer
You read what several produced ASYNCHRONOUS	1 : n	you sample; no time to check every detail

GOTCHA

At every ratio you have to learn to delegate: accept that you no longer control the details. You lead.

The two extreme ratios

What “AI for science” enthusiasm is usually about.

THE ONE SAID OUT LOUD

$0 : [1, n]$

Nobody in the loop

You state the question and meet the work finished, or not at all. It assumes a goal can be specified completely, and that checking the result is cheaper than producing it.

Exactly how human reviewers work with papers submitted to journals and conferences.

THE SAME DREAM FROM BEHIND

$n_H > n_{\text{lema}}$

Several of you per result

If results arrive faster than humans can read them, review is the bottleneck and the ratio inverts: a room of readers for one producer.

This happens once lemas start producing scientific artifacts alien to humans.

GOTCHA

Both need more than a good language model: without very good peripheral tooling, no lema works reliably far from a human agent.

PART 3

Working with lemas

The technologies to lead them

AGENTS.md

the instructions you were retyping

Per project. Read at session start, with nobody typing. Nothing enforces it.

WHERE IT LANDS

At the bottom of the system prompt, in every client.
Nobody typed it, and it is there every session.

Deliberately a *surface*: what the project is, how work is run, where everything lives, and the commands that must stay green. Anything longer than a line is a link to a skill.

Walk-through: youtu.be/68k5TgQwoLs

```
# AGENTS.md - maintaining colgrep-mcp

## What this is
## Execution model
## Load the dev plugin
## Repo map
## Gate commands
```

github.com/CrackingShells/colgrep-mcp/blob/main/AGENTS.md

GOTCHA

A document, not a mechanism: natural-language instructions may be ignored by a lema.

Skills

the procedures that travel between projects

When to use it, the steps, what the result should look like. Written for someone who has to follow your way of doing things.

START HERE

A one-file skill

A single `SKILL.md` with frontmatter. The frontmatter is what the agent always sees; the body loads when it matches.

[sysbio-curie/MCP_Hackaton](https://github.com/sysbio-curie/MCP_Hackaton)

THE CEILING

spawning-agent-plugins

Turns a repository into an installable plugin for three ecosystems. Ships references/, scripts/, assets/ and its own evals/.

[CrackingShells/cracking-shells-playbook/tree/main/skills/spawning-agent-plugins](https://github.com/CrackingShells/cracking-shells-playbook/tree/main/skills/spawning-agent-plugins)

GOTCHA

Like `AGENTS.md`, a skill rests primarily on natural language: it can be read and still not be followed. Only the scripts it carries are enforced.

Walk-through: youtu.be/68k5TgQwoLs

MCP servers

the call itself, declared

Three primitives, and two features that live inside a tool call. Five demo servers in the hackathon repository, each deliberately tiny.

Tools

One call, one return, declared with its types. The floor everyone knows.

[demo-tools](#)

TWO FEATURES OF A TOOL CALL

progress

Streams “14 of 200” and a log line from inside the call. Skip it and a long call goes dark.

[demo-progress](#)

elicitation

Missing parameters? The call stops and asks, typed. It does not guess.

[demo-elicitation](#)

Resources

Content with a URI, so the server can point at anything you can name.

[demo-resource](#)

Prompts

Parameterised templates: your side of the conversation, written once.

[demo-prompt](#)

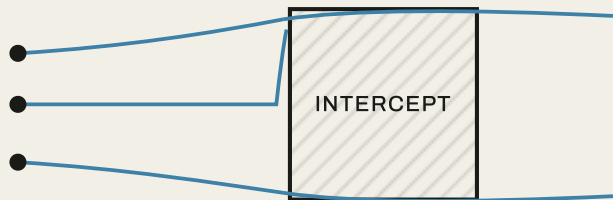
TIP

If you have the algorithm, make it a tool. Never spend natural language on what you can compute.

Hooks

a recurring pattern, intercepted at a fixed moment of the agent loop

Two conditions before you write one: the pattern repeats, and you can name the moment it can be caught. What happens after the interception is the part that varies.



The call and its arguments are visible before it runs. Everything the agent does passes a small number of such moments.

DENY

the thing you never want done stops being available at all, rather than discouraged

INFORM

a fact put in front of the agent at the moment it matters, with nobody typing it

CONNECT

reach a third party in reaction: a message to your team's chat, a ticket, a notification on your phone

CLOSE A LOOP

one agent publishes an issue, another is dispatched in its own worktree, and the cycle runs without you

TIP

Deny raw `grep` and `find` in favour of a semantic search over your own documents. A lema is trained on bash and will not know your tool exists until you force it on them. Worked example in colgrep-mcp/hooks/.

Where a hook can fire

EVENT	CAN REFUSE	WHAT IT IS GOOD FOR
PreToolUse	yes	the deny pattern; argument validation
PostToolUse	no	catch it after the fact; run the linter; append to a log
UserPromptSubmit	yes	the inject pattern; add context to every message you send
SessionStart	no	hand it the standing rules, the branch, today's dev log
SessionEnd	no	write the summary nobody will read but everybody needs
Stop · SubagentStop	no	check a helper's output before it reaches the primary
PreCompact · Notification	no	save what is about to be summarised away
...	...	...

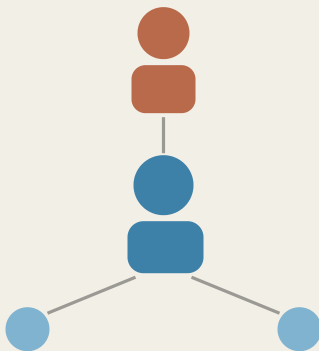
Event names follow Claude Code; other harnesses expose a subset under their own names. Hooks are the one thing here that does not package cleanly: the useful ones depend on your intended use — denying, injecting, monitoring, routing. Full event list, payloads and matchers: code.claude.com/docs/en/hooks

Lemas working together

n_{lema} goes up on its own

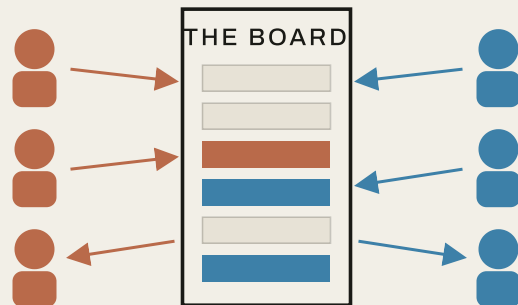
A helper starts from zero. What the primary wrote down is its whole world.

Today: a hierarchy



An interactive session needs one agent to talk to. Hence the pressure on language models as orchestrators.

At the async limit



One board. Several of you post; several lemas claim, work and return. Delegating to subagents happens out of view, so the hierarchy leaves your view, not the system.

Where agent behaviour is defined

two channels, and you already have both

IN THE SYSTEM PROMPT

AGENTS.md when the division of work is a standing property of the project: the execution model every session inherits, written once

agent file when a helper needs a role of its own: persona, tool diet, what it must refuse — subagents in Claude Code, its own form in every other harness

IN USER MESSAGES

skills when the division of work is complicated enough to need several files, or small scripts that let the agents check themselves

prompts when the same division of work has to be instructed repeatedly, tied to a state of the project you recognise

hooks when the division of work follows from a pattern the agent system can detect itself: every commit dispatches a reviewer

TIP

Tuning behaviour needs nothing new: every mechanism in this part composes through these two channels.

Keeping track of lemas' work

two audiences, two formats

FOR HUMANS

Reports

`writing-reports@cracking-shells`

A skill that fixes the shape of the output: what was decided, what was measured, what is still open. Model first, no code dumps.

TIP

Optimise the shape of reports through skills, so they are easy to read **for you**.

FOR LEMAS, BETWEEN THEMSELVES

History, and the reasoning

`git` the history, a named coordinate you can return to, and the only tool that points backwards

`logs` change logs, dev logs: the reasoning between two commits

TIP

Do not even try to read them. Just make sure they exist, and that the next session finds them.

Technology table

What should the **system as a whole** do? Then split it by what you trust to compute each part.

WHAT	WHEN	
AGENTS.md	the same instructions every session, per project	read at start; a document, not a mechanism
a skill	a procedure that must travel between projects	when to use something, and how
a tool	an algorithm exists	the model should <i>call</i> it, not imitate it
a resource	data you want reachable by name	one address you and the agent both use
a prompt	a message, or its template, you reuse all the time	your side of it, parameterised and written once
a hook	a recurring pattern, caught at a fixed moment	automating a response to an event
subagent files	a helper needs a role of its own	persona, tool diet, what it must refuse
communication surfaces	who may invoke whom	the division of work is clear, and communication costs less than a primary agent doing everything itself

PART 4

The hackathon

What exists, how to split it, what to pick

What already exists, by table

Every address below is in [inventory/mcp-servers.md](#). Nothing to copy down.

T3 7 topics

Genomics & omics tools

Well covered. One binary over ~30 sources: PubMed, ClinVar, ClinicalTrials, OncoKB, Reactome. Plus 23 single-database servers, several of which I have used.

WHERE IT LIVES

[github.com/genomoncology/biomcp](#)
`uv tool install biomcp-cli`
[github.com/Augmented-Nature](#)

T5 2 topics

Composing for a clinical question

Also covered, and your work is the composition, not the wrapping. Open Targets, PubMed and Synapse install as plugins with nothing to maintain.

WHERE IT LIVES

`/plugin marketplace add`
`anthropics/life-sciences`
15 connectors · 6 skills

T1 2 topics

Simulation & modelling engines

NeKo, MaBoSS and PhysiCell as three stateful servers, on the same SDK as the example servers. Heavy: a reference implementation to read, not a template. Open these four:

`session_manager.py` · `handoff.py`
`guidance.py` · `tools/` vs `services/`

WHERE IT LIVES

[github.com/marcorusc/](#)
[mcp-biomodelling-servers](#)
`uvx --from mcp-biomodelling-servers`
`mcp-neko-server`

T2 4 topics

Imaging & chromatin architecture

Nothing exists, official or otherwise. Yours would be the first.

T4 4 topics

Institutional data & tooling

Nothing exists by definition. Your cohort, your LIMS, your pipelines.

Gaps worth building *(according to Opus)*

Checked against GitHub, the MCP registries and the aggregator sites, September 2026.

TARGET	STATUS	WHAT IS MISSING
cBioPortal	still open	the official server queries a mirror; the one that hits the live API has no adoption
DepMap	still open	reads a downloaded CRISPR-screen cache, not the live API
MSigDB / GSEA	confirmed gap	one backend treats it as one signature source among several; no dedicated server
COSMIC · CellMinerCDB	confirmed gap	nothing of any kind
OmniPath / pypath	confirmed gap	NeKo reaches it; there is no general server
GINsim · BoolNet · CellNOpt · CoLoMoTo	confirmed gap	the Boolean ecosystem beyond MaBoSS, all four

Choosing a project

Six to eight hours of real work. That is the whole budget.

Ask yourself first

Do you want to **build**?

Then pick the smallest thing that is genuinely yours:
one tool around one analysis your group actually runs.

Do you want to **learn**?

Then pick something too big on purpose, and accept
you may leave with insight instead of a server.

Both are legitimate, and they lead to different projects.
Decide before you sit down at a table.

Then split your own work four ways

- | | |
|-------------------------------|---|
| COMPUTATION | a right answer exists → a tool |
| INFERENCE | you are choosing to rely on the model → a skill |
| KNOWLEDGE YOU KEEP | the judgement that stays with you → nothing to write |
| KNOWLEDGE YOU DELEGATE | the part you are willing to hand over → and how you will check it |

TIP

If you cannot say what a right answer looks like,
you are writing a skill, not a tool.

How you present

7 minutes of presentation

3 minutes of questions

One slide on the topic, then screenshots or a live demo. If it is live, bring the screenshots anyway.

WHERE YOUR PROJECT LANDS

Open a pull request adding your repository as a git submodule under:

hackathon-projects/

github.com/sysbio-curie/MCP_Hackaton

That collection is what the joint paper is written from.
Everyone who submits is a co-author.